Say, we have taken 3 examples. They are all inquiries about ground transportation in Boston. And by visual inspection, we can see that their embedding patterns are very similar.

→ Here the model was able to capture the context & meaning of each piece of text. & then represents them as embeddings. Each dimension of an embedding, called a feature, represents a certain universal characteristic of text according to how the model understands it.

→ How is this possible? A LLM has been pre-trained with a vast amount of text data, where the training objective is set up in such a way as to encourage the model to extract contextual info^n about a piece of text & store it as embeddings.

⇒ Step④: Visualize Embeddings on a 2D Plot.

We can investigate this further by compressing the embeddings to 2 dimensions & plotting them on a scatter plot. What we would expect is that texts of similar meaning would be closer to each other & vice versa.

↳ Do note that as we compress the embeddings to lower dimensions, the info^n retained becomes lesser. However, humans can only visualize in 2D or 3D, & it turns out this is still a good enough approximat^n to help us gain intuition about data.

↳ By visual inspection, we can see that texts of similar meaning are indeed located close together. We see inquiries about tickets on the left, inquiries about airlines somewhere around the middle & inquiries about ground transportat^n on the top right.

These kinds of insights enable various downstream analyses & app's, such as topic modeling by clustering documents into groups. In other words, text embeddings allow us to take a huge corpus of unstructured text & turn it into a structured form, making it possible to objectively compare, dissect & derive insights from all that text.

→ Text embeddings make possible a wide array of downstream applicat^ns such as semantic search, clustering & classification.

⊛ Semantic Search

Here, you'll learn how to use text embeddings to search for the answer to a given query among the sentences in a dataset. Since the embeddings takes semantics into account, this process is called semantic search.

We deal with unstructured text data on a regular basis, and one of the common needs is to search for info^n from a vast repository. A common approach is keyword-matching, but the prob^m with this is that the results are limited to the exact query entered.

This is where we can utilize text embeddings. Embedding can capture the meaning of a piece of text beyond keyword-matching.

⇒ Step-by-Step G

Let's use same 9
a list of Frequentl
new query comes in,
closest FAQ, so w
Here is the list a

- which airlines
- show me the
- show me round
- i'd like th
- show me a
- show me b
- of all airli
- what grou
- i would
- on sept
- which

↳ Let's say a
taxi or a bu
the "taxi" &
in our FAQ,
semantic se

⇒ Step-by-Step Guide

Let's use same 9 data points & pretend that those make up a list of Frequently Asked Questions (FAQ). Whenever a new query comes in, we want to match that query to the closest FAQ, so we can provide the most relevant answer.

Here is the list again:

- which airlines fly from boston to washington dc via other cities
- show me the airlines that fly b/w toronto & denver
- show me round trip first class tickets from newyork to miami
- i'd like the lowest fare from denver to pittsburgh
- show me a list of ground transportation at boston airport
- show me boston ground transportation
- of all airlines which airline has the most arrivals in atlanta
- what ground transportation is available in boston
- i would like your rates b/w atlanta & boston on september thirt
- which airlines fly b/w boston & pittsburgh.

↳ Let's say a person enters the query "How can I find a taxi or a bus when the plane lands ?". Note that the "taxi" & "bus" keywords don't exist anywhere in our FAQs, so let's see what results we get with semantic search.

→ Implementation-wise, there are many ways we can approach this. And in our case, we use cosine similarity to compare the embeddings of the search query with those from the FAQs & find the most similar ones.

↳ Step①: Embed the Documents.

The 1st step is to turn the documents into embeddings. We embed each inquiry by calling Cohere's Embed endpoint with co.embed( ). It takes in texts as input & returns embeddings as output. [Check the Previous ~~event~~ pages]

↳ Step②: Embed the Search Query.

Next, we embed the query using the same get_embeddings( ) function. But now we set search-query as the input_type because we're now embedding the search query

```
#Define new query
new_query = "How can I find a taxi or a bus when
             the plane lands?"

# Get embeddings of the new query.
new_query_embeds = get_embeddings ( [new_query],
                    input_type = "search_query")
```

↳ Step③: Perfe

Next, we create
similarity to dete
is to the query

↳ Below are the
similarity score
The top-3 ranke
transportation,
options. Notice
"taxi" or "bu
keywords "gr
similar in me
capture this.

↳ Query:
How can I
Most Similar
Similarity: 0
Similarity:
Similarity:

↳ Step④
We can al
did earlier &
ground tra
The quer
to document

↳ Step③ : Perform Search

Next, we create a function get-similarity() that uses cosine similarity to determine how similar each of the documents is to the query.

↳ Below are the results, showing the FAQs with their similarity score (ranging from 0 to 1; higher scores are better). The top-3 ranked FAQs we get are inquiries about ground transportation, which are very relevant considering the other options. Notice that they don't contain the keywords "taxi" or "bus" & search query doesn't contain the keywords "ground transportation". But they are the most similar in meaning compared to the rest & their embeddings capture this.

↳ Query:

How can I find a taxi or a bus when the plane lands?

Most Similar Documents:

Similarity: 0.37; show me a list of ground transportation at boston (airport)

Similarity: 0.36; what ground transportat" is available in boston

Similarity: 0.33; show me boston ground transportation

↳ Step④ Visualize the Results in a 2D Plot

We can also plot this new query on a 2D plot as we did earlier & the query is located closest to the FAQs about ground transportation.

The query about "a taxi or a bus" is located closest to documents about ground transportation.

⇒ Text Clustering

Here, you'll leverage embeddings & K-Means clustering to apply a text dataset into different clusters with semantically similar sentences.

As the amount of unstructured text data increases, organizat will want to be able to derive an understanding of its conte One example would be to discover underlying topics in a collection of documents, so we can explore trends & insights Another could be for businesses to segment customers based on preferences & activity.

↳ These kinds of tasks fall under a category called clustering In ML, clustering is a process of grouping similar documents into clusters. It is used to organize a large no. of documents into a small no. of groups. And it lets us discover emerging patterns in a collection of documents w/o us having to specify much info^n beyond supplying the data.

⇒ Step-by-Step Guide

↳ Consider the same 9 data points as prev:

- which airlines fly . -

— —

↳ Step ①: Embed the Text for Clustering

We embed the documents using the same get_embeddings() function as before, but now we set input_type="clustering" because we'll use the embeddings for clustering.

↳ Step ② Cluster

Implementation-wise, u

these data points.

Other than provid

key info^n we need to

of clusters we want t

actual app^n's, but sin

the no. of clusters

Pick the

— n-clusters

— # Cluster

kmeans_m

classes =

store

df-clu

df-cl

↳ Step ③ Visuali

The plot below sh

It looks to be spot

info^n & one cluste

⇒ • show me a

• which airline

↳ Step ② Cluster the Embeddings

Implementation-wise, we use the K-means algo^{m}s to cluster these data points.

Other than providing the embeddings, the only other key info^m we need to provide for the algo^m is the no. of clusters we want to find. This is normally larger in actual app^{n}s, but since our dataset is small, we'll set the no. of clusters to 2.

```
# Pick the no. of clusters
n_clusters = 2
# cluster the embeddings
kmeans_model = KMeans(n_clusters=n_clusters,
                      random_state=0)
classes = kmeans_model.fit_predict(embeds).tolist()

# store the cluster assignments.
df_clust = df_pc2.copy()
df_clust['cluster'] = (list(map(str, classes)))
```

↳ Step ③ Visualize the Results in a 2D Plot

The plot below shows the clusters that the algo^m returned. It looks to be spot on, where we have one cluster related to airline info^m & one cluster related to ground service info^n. Each cluster corresponds to a particular topic.

⇒ • show me airlines that fly b/w x & y.
• which airline fly from boston to -

• show me a list of ground transportation
• show me boston ground transportation

⇒ Text Classification

Previously we saw how to use embeddings to enable use cases such as semantic search, clustering & topic modelling. All of these are "unsupervised learning algo's", where we don't know the no. of groups & what they are.

But what if we already know the groups (or classes) that we want to group our documents into? Here, you'll learn about classificat^n which is a "supervised learning algo^m" where we know the specific classes we want to use.

Like all ML models, classificat^n models need to be evaluated in order to improve & measure their performance. Here, you'll learn 4 diff. ways to evaluate classificat^n models.

↳ Types of Text Classification

A classification task falls under one of these 2 categories

- Binary classificat^n → # classes = 2

  Ex. - A spam classifier could assign emails one of the two classes: "Spam" or "Not Spam".

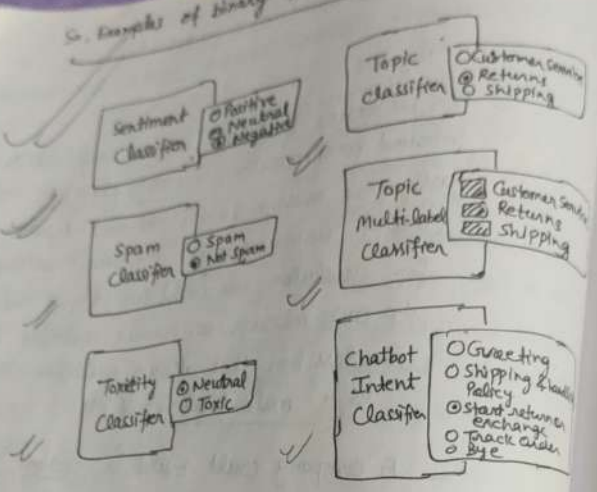
  - An online forum could use a toxicity classifier to assist with content moderation by classifying posts as "Neutral" or "Toxic".

- Multi-class classification → # classes > 2

Ex.

- A restaurant could use a sentiment classifier to categorize customer reviews as "Positive", "Neutral", or "Negative".
- For businesses that deal with a large volume of inbound communication, a topic classifier could assign the message to the relevant department (one of "Customer Service", "Returns", or "Shipping") for resolution. We could assign just one label to each message, or permit multiple labels. A classifier that allows multiple labels is a multi-label classifier.
- A company could build a chatbot intent classifier to detect the intent behind a user's message (one of "Greeting", "Shipping and handling policy", "Start return or exchange", "Track order", or "Bye"). Intent classifiers are broader than topic classifiers, as they include intents like "Greeting" or "Bye" that enable them to respond appropriately when a user says "hi" or "bye", for example.

5. Examples of binary & multi-class classifications



→ Evaluation matrices.

↳ 4 most imp. ways ⇒ Accuracy, Precision, Recall & F1.

$$\frac{\text{No. of Correct Predictions}}{\text{Total no. of Predictions}}$$

↳ Accuracy is not a good metric for Imbalanced data, a scenario when certain classes contain way more data points than the others. If classifier had not been learning anything & was simply classifying all the outputs to be 0 (not spam), we would get 17 out of 20 correct classifications, which translates to a very high accuracy of 85%! Clearly, something is not right.

→ If you have not n
we have way more e
that are spam.
→ Accuracy can so
datasets. The reaso
true negative categ
the rest.
So, even if th
the other 3 categ
good, maski

The
Acc

↳ Precision, Re
view of a cla
focus on True Negat

↳ B/w Precision & R
hard to optimize for
the FP (thereby im
of the FN
(False

↳ Which then brings
Prioritize – Prec